

Modeling Song Popularity with Neural Networks and Bayesian Regression

DS4420

Prepared by: Siyuan Chen, Yifan Wang

Instructor: Dr. Eric Gerber

Date: April 16, 2026

Introduction

Predicting song popularity is an important problem in the music industry, as it can help platforms and artists better understand listener preferences and optimize content recommendations. With the increasing availability of large-scale music data, machine learning methods provide new opportunities to explore the relationship between audio characteristics and audience engagement.

This project investigates whether a small set of audio features can be used to predict song popularity. Using a Spotify dataset containing over 100,000 tracks, we focus on five key attributes: danceability, energy, tempo, loudness, and valence. These features capture different aspects of a song's structure and emotional content.

Two modeling approaches are applied to address this problem. A neural network is implemented to capture potential non-linear relationships and evaluate predictive performance. In parallel, a Bayesian regression model is used to provide interpretable estimates and quantify uncertainty in the relationship between features and popularity.

By comparing these two approaches, this study aims to assess both the predictive power of audio features and the extent to which they can explain variation in song popularity. The results highlight the strengths and limitations of each method and provide insight into the role of audio characteristics in determining music success.

Data Section

The dataset used in this project is the Spotify Tracks Dataset, a publicly available collection containing approximately 114,000 songs across a wide range of genres. Each track includes 21 attributes covering audio features, metadata, and a popularity score assigned by Spotify.

Target Variable

The target variable is popularity, an integer score ranging from 0 to 100 that reflects a track’s recent streaming performance and listener engagement on the Spotify platform. The distribution is right-skewed, with a mean of approximately 33, indicating that most tracks have relatively low popularity.

Selected Features

Five continuous audio features were selected as predictors:

Feature	Description
 danceability	How suitable a track is for dancing (0.0–1.0)
 energy	Perceptual measure of intensity and activity (0.0–1.0)
 tempo	Estimated beats per minute
 loudness	Overall loudness in decibels
 valence	Musical positiveness conveyed by the track (0.0–1.0)

These features capture different aspects of a song’s structure and sound, and serve as inputs for the machine learning models.

Preprocessing

Rows with missing values were removed. No data loss occurred, as the selected features contained no null entries. The dataset was then split into 80% training and 20% test sets using random sampling.

To ensure stable gradient-based training, all features were standardized using the mean and standard deviation of the training set. The same transformation was applied to the test set to prevent data leakage. This step ensures that all input

variables are on a comparable scale and improves the convergence of the neural network.

Neural Network Section

Model Architecture

A multilayer perceptron (MLP) was implemented from scratch using NumPy to predict track popularity from the selected audio features. The network consists of an input layer with 5 nodes, followed by two hidden layers with 64 and 32 nodes, and a single output node for regression:

$$5 \rightarrow 64 \rightarrow 32 \rightarrow 1$$

ReLU activation was applied to both hidden layers to introduce non-linearity. The output layer uses a linear activation function, which is appropriate for continuous prediction tasks.

Implementation

The model was implemented entirely without pre-built machine learning libraries. Several core components were constructed manually:

Weight Initialization: He initialization ($w \sim N(0, \sqrt{2/n_{in}})$) was used to stabilize gradients in networks with ReLU activations.

Activation Function: ReLU ($\max(0, z)$) was applied to hidden layers.

Forward Pass: Inputs are propagated through each layer via matrix multiplication, followed by bias addition and activation.

Backpropagation: Gradients of the mean squared error (MSE) loss are computed analytically and propagated backward using the chain rule.

Optimizer: Mini-batch stochastic gradient descent was used with a batch size of 256 and a learning rate of 0.001. The training data is shuffled at each epoch to reduce gradient variance.

Training

The model was trained for 500 epochs. Mean squared error (MSE) was recorded after each epoch to track training progress. The loss curve shows a consistent downward trend, indicating that the network successfully learns from the data and converges over time.

Results (Neural Network)

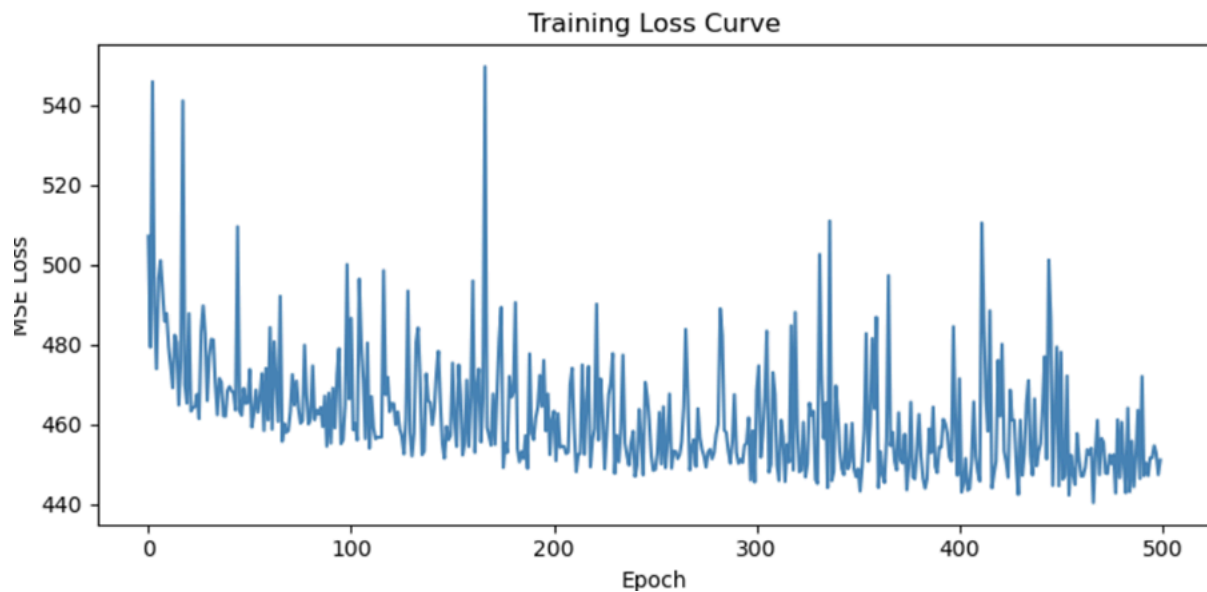
Quantitative Performance

The manually implemented MLP was evaluated on the held-out test set (20% of the data). The results are as follows:

Metric	Value
RMSE	21.52
MAE	17.59
R ²	0.071

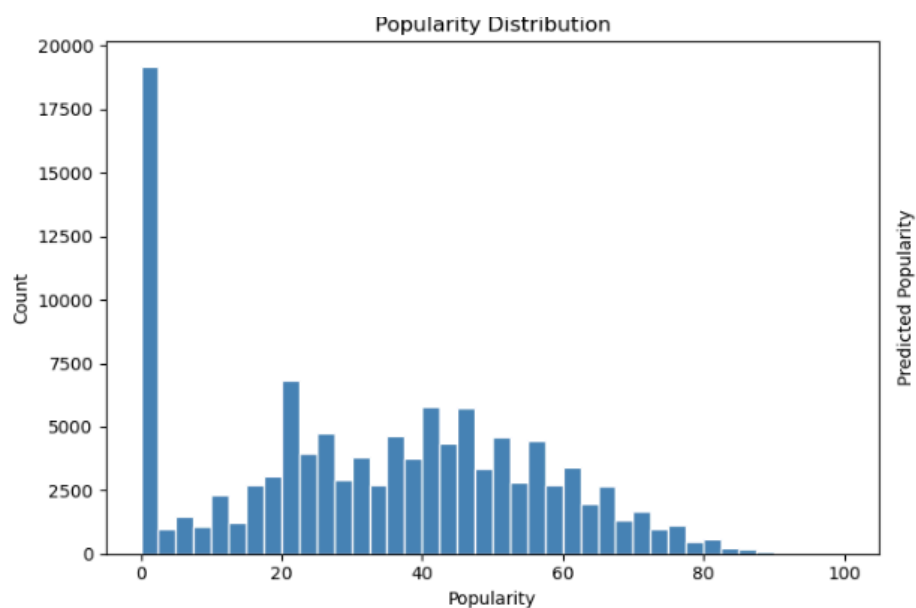
The model achieves a mean absolute error of approximately 17.6 popularity points on a 0–100 scale. The low R² of 0.071 indicates that the five audio features account for only about 7% of the variance in popularity, suggesting that audio characteristics alone are insufficient to fully explain a track's commercial performance.

Training Loss Curve



The training loss curve shows MSE decreasing from approximately 550 at epoch 1 to around 445 by epoch 500. The high variance throughout training is expected behavior for mini-batch SGD, as each parameter update is computed on a random subset of the data rather than the full training set. The overall downward trend confirms that the network successfully converges and learns a meaningful mapping from audio features to popularity.

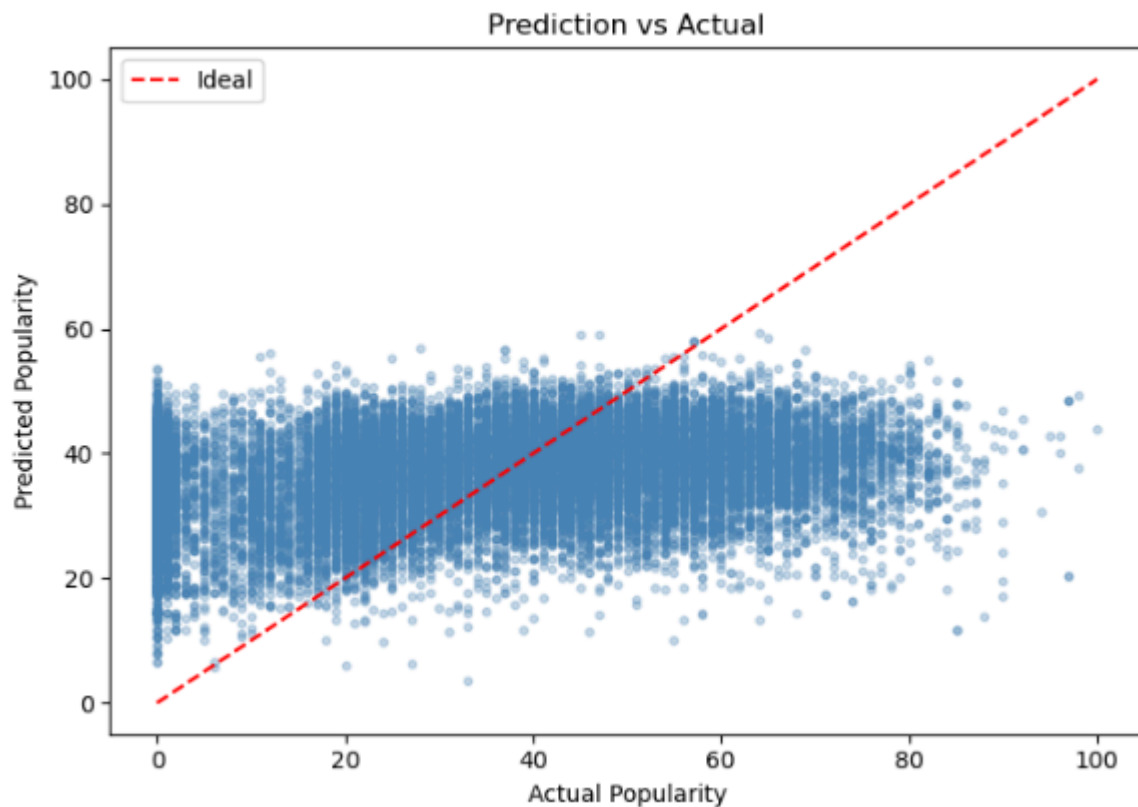
Popularity Distribution



The popularity distribution reveals a strongly right-skewed target variable with a large concentration of tracks near zero. This imbalanced distribution presents a challenge

for regression models, as the model is implicitly penalized more for errors in the dense low-popularity region, which may cause it to bias predictions toward the mean.

Prediction vs. Actual



The scatter plot shows that predictions are compressed into a narrow band of roughly 15–55, while actual popularity spans the full range of 0–100. The model fails to capture tracks with very low (near 0) or very high (above 70) popularity scores. This pattern is consistent with regression toward the mean, a known limitation of models with low explanatory power. The divergence from the ideal diagonal line (shown in red) further confirms that the selected audio features are weak predictors of popularity in isolation.

The limited predictive performance is not unexpected. Music popularity is driven by factors beyond audio content, including artist reputation, social media exposure, playlist placement, and release timing — none of which are captured in this dataset. The neural network nonetheless provides a useful baseline and demonstrates that

audio features carry some predictive signal, as evidenced by the positive R^2 and the general upward trend visible in the scatter plot.

Bayesian Regression Section

Using the preprocessed dataset described above, we applied a Bayesian regression model to further analyze the relationship between audio features and song popularity.

Bayesian Model Section

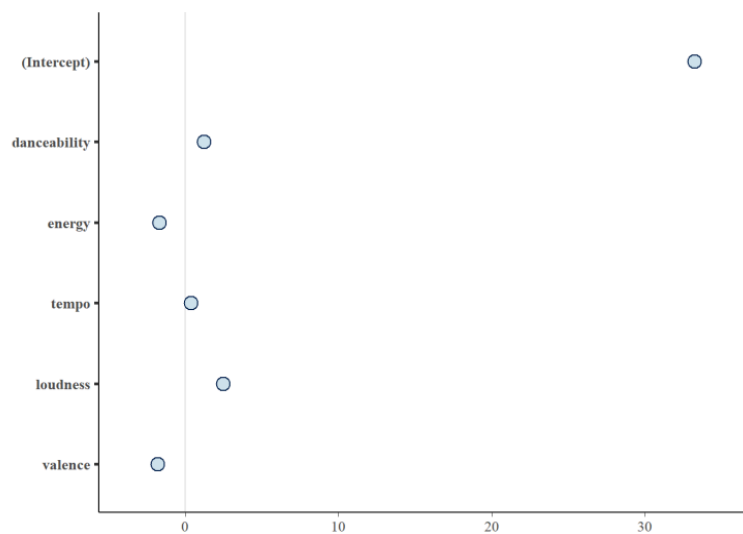
We used a Bayesian regression model in R to predict Spotify track popularity based on several audio features, including danceability, energy, tempo, loudness, and valence. One advantage of using a Bayesian approach is that it not only gives us coefficient estimates, but also shows the uncertainty of each estimate through credible intervals. This makes it easier to understand how each feature affects popularity. Before training the model, we standardized all input features so that they are on the same scale and easier to compare.

Bayesian Interpretation

From the model results, we can see that different audio features have different effects on song popularity. Loudness and danceability have positive effects, which means songs that are louder and more danceable tend to be more popular. On the other hand, energy and valence show negative relationships with popularity, suggesting that higher values in these features do not always lead to higher popularity. Tempo has a positive effect, but its impact is relatively small compared to the other features. The 95% credible intervals for all variables do not include zero, which suggests that these features have meaningful effects on popularity. Overall, loudness appears to be the most important feature based on its larger coefficient value.

Figure 1: Posterior Coefficient Intervals for Bayesian Model

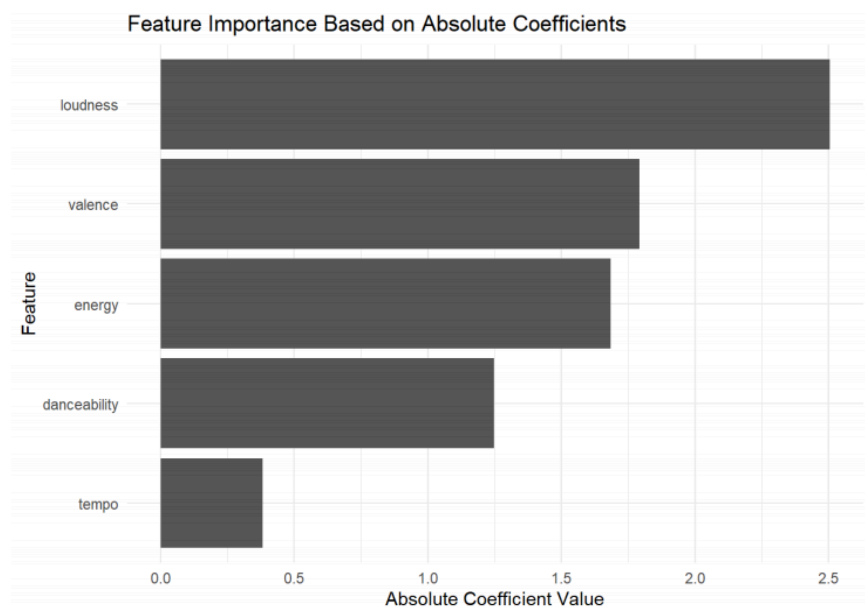
This figure shows the posterior distributions and credible intervals for each coefficient.



This plot shows the estimated coefficients and their credible intervals for each feature. Loudness and danceability have positive effects on popularity, while energy and valence have negative effects. All intervals do not include zero, suggesting that these features have meaningful impacts.

Figure 2: Feature Importance Based on Absolute Coefficients

This figure highlights the relative importance of each feature based on the magnitude of the coefficients.



This plot compares the relative importance of each feature based on the absolute value of the coefficients. Loudness appears to be the most important factor, followed by valence and energy. Tempo has the smallest impact among the selected features.

Model Comparison

The neural network and Bayesian regression models reveal consistent but limited predictive relationships between audio features and song popularity.

From a predictive standpoint, the neural network achieves modest performance, with an R^2 of 0.071 and predictions concentrated in a narrow range of approximately 15–55, as shown in the prediction vs. actual plot. This compression indicates that the model struggles to capture extreme values, particularly highly popular or very unpopular tracks. The observed regression toward the mean suggests that the selected audio features provide only weak predictive signals.

In contrast, the Bayesian regression model does not focus on prediction accuracy but instead provides interpretable insights into feature effects. The posterior estimates show that **loudness and danceability have positive effects**, while **energy and valence have negative relationships with popularity**. The 95% credible intervals for all coefficients exclude zero, indicating that these effects are statistically meaningful despite the overall low explanatory power.

Importantly, both models lead to consistent conclusions. While the neural network captures a general upward trend in the data, the Bayesian model confirms that certain features have systematic effects on popularity. However, both approaches highlight the same limitation: audio features alone explain only a small portion of the variation in popularity.

Overall, the neural network provides flexibility in capturing non-linear patterns but suffers from low interpretability and limited predictive accuracy. The Bayesian model offers clear interpretation and uncertainty quantification, making it more suitable for understanding feature relationships. Together, the two models provide complementary insights, combining predictive modeling with statistical explanation.

Conclusion

This project investigated whether audio features can be used to predict song popularity using both a neural network and a Bayesian regression model. The results show that while these features contain some predictive signal, they are not sufficient to fully explain variation in popularity.

The neural network model demonstrates limited predictive performance, with a low R^2 value and a tendency to predict values within a narrow range. This suggests that the model is unable to capture extreme popularity outcomes. At the same time, the Bayesian regression model provides clear and interpretable relationships between features and popularity. Variables such as loudness and danceability show consistent positive effects, while energy and valence exhibit negative relationships.

Despite differences in methodology, both models lead to a similar conclusion. Audio features alone explain only a small portion of song popularity. External factors such as artist reputation, marketing exposure, and platform dynamics likely play a much larger role.

Overall, this project highlights the value of combining predictive and interpretable models. The neural network captures general patterns in the data, while the Bayesian approach provides insight into feature effects and uncertainty. Together, they offer a more complete understanding of the relationship between audio characteristics and popularity.

Appendix

Neural Network Implementation

1. Code

```
class ManualMLP:
    """
    Multi-Layer Perceptron for regression, implemented from scratch using NumPy.
    Architecture : input -> hidden layers (ReLU) -> output (linear)
    Optimizer      : mini-batch stochastic gradient descent
    """

    def __init__(self, layer_sizes, learning_rate=0.001,
                  max_iter=500, batch_size=256, random_state=42):
        np.random.seed(random_state)
        self.layer_sizes = layer_sizes
        self.lr = learning_rate
        self.max_iter = max_iter
        self.batch_size = batch_size
        self.loss_history = []

        # He initialization (suited for ReLU activations)
        self.weights = []
        self.biases = []
        for i in range(len(layer_sizes) - 1):
            w = np.random.randn(layer_sizes[i], layer_sizes[i+1]) * np.sqrt(2.0 / layer_sizes[i])
            b = np.zeros((1, layer_sizes[i+1]))
            self.weights.append(w)
            self.biases.append(b)

        # Activation functions
        def _relu(self, z):
            return np.maximum(0, z)

        def _relu_grad(self, z):
            return (z > 0).astype(float)

        # Forward pass
        def _forward(self, X):
            self._activations = [X]
            self._z_vals = []
            for i, (w, b) in enumerate(zip(self.weights, self.biases)):
                z = self._activations[-1] @ w + b
                self._z_vals.append(z)

                a = self._relu(z) if i < len(self.weights) - 1 else z
                self._activations.append(a)
            return self._activations[-1]
```

```

# Backward pass (backpropagation)
def _backward(self, y_true):
    m = y_true.shape[0]
    y_true = y_true.reshape(-1, 1)

    delta = 2 * (self._activations[-1] - y_true) / m

    grad_w = [None] * len(self.weights)
    grad_b = [None] * len(self.weights)

    for i in reversed(range(len(self.weights))):
        grad_w[i] = self._activations[i].T @ delta
        grad_b[i] = delta.sum(axis=0, keepdims=True)
        if i > 0:
            delta = (delta @ self.weights[i].T) * self._relu_grad(self._z_vals[i-1])

    # Gradient descent weight update
    for i in range(len(self.weights)):
        self.weights[i] -= self.lr * grad_w[i]
        self.biases[i] -= self.lr * grad_b[i]

# Training loop
def fit(self, X, y):
    for epoch in range(self.max_iter):
        perm = np.random.permutation(X.shape[0])
        X_s, y_s = X[perm], y[perm]

        for start in range(0, X.shape[0], self.batch_size):
            Xb = X_s[start : start + self.batch_size]
            yb = y_s[start : start + self.batch_size]
            self._forward(Xb)
            self._backward(yb)

        loss = np.mean((self._forward(X).flatten() - y) ** 2)
        self.loss_history.append(loss)

        if (epoch + 1) % 100 == 0:
            print(f"Epoch {epoch+1:4d}/{self.max_iter} MSE Loss: {loss:.4f}")

# Prediction
def predict(self, X):
    return self._forward(X).flatten()

```

2.model summary

Item	Detail
Model Type	Multilayer Perceptron (Regression)
Implementation	From scratch — NumPy only
Input Features	danceability, energy, tempo, loudness, valence
Target	popularity (0–100)
Architecture	5 → 64 → 32 → 1
Hidden Activation	ReLU
Output Activation	Linear
Weight Initialization	He initialization
Optimizer	Mini-batch SGD
Learning Rate	0.001
Batch Size	256
Epochs	500
Train/Test Split	80% / 20%
Standardization	Manual (zero mean, unit variance)
RMSE	21.52
MAE	17.59
R ²	0.071

Bayesian Model Code (R)

1.Code

```
# Load packages
library(rstanarm)
library(bayesplot)
library(ggplot2)

# Read data
data <- read.csv("dataset.csv")

# Keep only the variables needed for the Bayesian model
df <- data[, c("popularity", "danceability", "energy", "tempo", "loudness", "valence")]

# Remove missing values
df <- na.omit(df)

# Standardize predictors
predictor_cols <- c("danceability", "energy", "tempo", "loudness", "valence")
df[predictor_cols] <- scale(df[predictor_cols])

# Fit Bayesian regression model
bayes_model <- stan_glm(
  popularity ~ danceability + energy + tempo + loudness + valence,
  data = df,
  family = gaussian(),
  chains = 4,
  iter = 2000,
  seed = 123,
  refresh = 0
)

# Model summary
summary(bayes_model)
```

2. Model Summary

```
##
## Model Info:
## function:      stan_glm
## family:        gaussian [identity]
## formula:       popularity ~ danceability + energy + tempo + loudness + valence
## algorithm:     sampling
## sample:        4000 (posterior sample size)
## priors:        see help('prior_summary')
## observations:  114000
## predictors:    6
##
```



```
## Estimates:
##           mean    sd   10%   50%   90%
## (Intercept) 33.2    0.1  33.2  33.2  33.3
## danceability 1.2    0.1   1.1   1.2   1.3
## energy      -1.7    0.1  -1.8  -1.7  -1.5
## tempo        0.4    0.1   0.3   0.4   0.5
## loudness     2.5    0.1   2.4   2.5   2.6
## valence     -1.8    0.1  -1.9  -1.8  -1.7
## sigma       22.2    0.0  22.1  22.2  22.2
##
## Fit Diagnostics:
##           mean    sd   10%   50%   90%
## mean_PPD 33.2    0.1  33.1  33.2  33.4
##
## The mean_ppd is the sample average posterior predictive distribution of the outcome variable
## (for details see help('summary.stanreg')).
##
## MCMC diagnostics
##           mcse Rhat n_eff
## (Intercept) 0.0  1.0  1486
## danceability 0.0  1.0  4063
## energy       0.0  1.0  2790
## tempo        0.0  1.0  5731
## loudness     0.0  1.0  2807
## valence      0.0  1.0  3647
## sigma        0.0  1.0  3228
## mean_PPD     0.0  1.0  2315
## log-posterior 0.1  1.0  1551
##
## For each parameter, mcse is Monte Carlo standard error, n_eff is a crude measure of effective
## sample size, and Rhat is the potential scale reduction factor on split chains (at convergence
## Rhat=1).
```

3. Credible Intervals

```
##           2.5%    97.5%
## (Intercept) 33.1092966 33.3728500
## danceability 1.0866201 1.3973424
## energy      -1.8769371 -1.4720531
## tempo        0.2514984 0.5165489
## loudness     2.2997594 2.7103400
## valence     -1.9497952 -1.6357636
## sigma       22.0923589 22.2734321
```